Apriori and FP-Growth: the same baskets, two routes

An executable, from-scratch lesson in frequent-itemset mining and association rules.

Run: open this notebook in Jupyter and choose **Kernel → Restart Kernel and Run All Cells**. It needs Python 3.9+ and IPython (included with a normal Jupyter installation); no data downloads, mining packages, Graphviz, or plotting libraries are needed. Outputs are already saved for reading without execution.

You will count frequencies, watch an FP-tree grow one transaction at a time, inspect conditional pattern bases and trees, follow Apriori's join/prune/count steps, compare exact results, generate rules, and measure runtime. The readable implementations prioritize teaching over production performance.

In [1]:
```
from collections import Counter, defaultdict
from itertools import combinations
from math import ceil, isclose
from statistics import median
from time import perf_counter_ns
from html import escape
import platform, random, sys
from IPython.display import display, HTML, SVG, Markdown
import os
import json

def label(items):
    return "{" + ", ".join(sorted(items)) + "}"

def table(headers, rows):
    html = '<table style="border-collapse:collapse">'
    html += '<tr>' + ''.join('<th style="padding:6px;text-align:left">'+esca
    for row in rows:
        html += '<tr>' + ''.join('<td style="padding:6px;border-top:1px soli
    display(HTML(html + '</table>'))


# Used unless BASKETS_JSON is supplied.
default_baskets = [
    ["bread", "milk"],
    ["bread", "diaper", "beer", "eggs"],
    ["milk", "diaper", "beer", "cola"],
    ["bread", "milk", "diaper", "beer"],
    ["bread", "milk", "diaper", "cola"],
]

raw_baskets = json.loads(
    os.environ.get("BASKETS_JSON", json.dumps(default_baskets))
)
```

```
if (
    not isinstance(raw_baskets, list)
    or not raw_baskets
    or any(
        not isinstance(basket, list)
        or any(not isinstance(item, str) for item in basket)
        for basket in raw_baskets
    )
):
    raise ValueError("BASKETS_JSON must be a nonempty list of lists of strir

baskets = [set(basket) for basket in raw_baskets]

MIN_SUPPORT = float(os.environ.get("MIN_SUPPORT", "0.6")) # Default exercise
MIN_CONFIDENCE = float(os.environ.get("MIN_CONFIDENCE", "0.75")) # Default e
basket_multiplier = int(os.environ.get("BASKET_MULTIPLIER", "200")) # Defaul

if not 0 < MIN_SUPPORT <= 1:
    raise ValueError("MIN_SUPPORT must be greater than 0 and at most 1.")
if not 0 <= MIN_CONFIDENCE <= 1:
    raise ValueError("MIN_CONFIDENCE must be between 0 and 1.")
if basket_multiplier < 1:
    raise ValueError("BASKET_MULTIPLIER must be a positive integer.")

N = len(baskets)
MIN_COUNT = ceil(MIN_SUPPORT * N)

# Compare baskets without depending on their order,
# while preserving the number of repeated baskets.
IS_BASELINE = (
    isclose(MIN_SUPPORT, 0.60)
    and isclose(MIN_CONFIDENCE, 0.75)
    and Counter(frozenset(b) for b in baskets)
        == Counter(frozenset(b) for b in default_baskets)
)

'''
# Include the settings in every exported PDF.
display(Markdown(
    f"### Exercise settings\n"
    f"- Baskets: **{N}**\n"
    f"- Minimum support: **{MIN_SUPPORT:.0%}**\n"
    f"- Minimum count: **{MIN_COUNT}**\n"
    f"- Minimum confidence: **{MIN_CONFIDENCE:.0%}**\n"
    f"- Benchmark basket multiplier: **{basket_multiplier}**"
))'''

display(Markdown(
    f"## {os.environ.get('EXERCISE_TITLE', 'Apriori and FP-Growth')}\n\n"
    f"- **Minimum support:** {MIN_SUPPORT:.0%}\n"
    f"- **Minimum count:** {MIN_COUNT}\n"
    f"- **Minimum confidence:** {MIN_CONFIDENCE:.0%}\n"
    f"- **Baskets:** {N}\n"
    f"- **Benchmark basket multiplier:** {basket_multiplier}"
))
```

```
# assert MIN_COUNT == 3 # This is only valid with the original exercice. The
print(f"\n{N} baskets; minimum support {MIN_SUPPORT:.0%} means at least {MIN
table(["Basket", "Items"], [(f"T{i}", label(t)) for i,t in enumerate(baskets
```

Exercise 2 — add an eggs-only basket

- **Minimum support:** 60%
- **Minimum count:** 4
- **Minimum confidence:** 75%
- **Baskets:** 6
- **Benchmark basket multiplier:** 200

```
6 baskets; minimum support 60% means at least 4 baskets.
```

Basket	Items
T1	{bread, milk}
T2	{beer, bread, diaper, eggs}
T3	{beer, cola, diaper, milk}
T4	{beer, bread, diaper, milk}
T5	{bread, cola, diaper, milk}
T6	{eggs}

1. Support and the first scan

For an itemset X, its **count** is the number of baskets containing every item in X. Its **support** is $count(X)/N$. A frequent itemset meets the inclusive support threshold.

FP-Growth's first scan counts individual items. Remove infrequent items and establish a single global order: decreasing frequency, then alphabetical order to break ties. Ordering every basket by its own original order would lose shared prefixes. The table below exposes the cumulative count after each basket.

In [2]:
```
counts = Counter()
all_items = sorted(set().union(*baskets))
scan_rows = []
for i, basket in enumerate(baskets, 1):
    # Count: counts.update(basket) counts how many baskets contain each item
    counts.update(basket)
    scan_rows.append([f"After T{i}"] + [counts[x] for x in all_items])
table(["First scan"] + all_items, scan_rows)

# Filter: counts[x] >= MIN_COUNT keeps items appearing in at least MIN_COUNT
order = sorted((x for x in counts if counts[x] >= MIN_COUNT), key=lambda x:

# Sort: every filtered basket follows the same global order: bread → diaper
rank = {x:i for i,x in enumerate(order)}
```

```
ordered_baskets = [sorted((x for x in t if x in rank), key=rank.get) for t i
table(["Item", "Count", "Support", "Keep?"],
      [(x, counts[x], f"{counts[x]/N:.0%}", counts[x] >= MIN_COUNT) for x ir
print("Global order:", " → ".join(order))
table(["Basket", "Filtered and ordered"], [(f"T{i}", " → ".join(t)) for i,t
```

First scan	beer	bread	cola	diaper	eggs	milk
After T1	0	1	0	0	0	1
After T2	1	2	0	1	1	1
After T3	2	2	1	2	1	2
After T4	3	3	1	3	1	3
After T5	3	4	2	4	1	4
After T6	3	4	2	4	2	4

Item	Count	Support	Keep?
bread	4	67%	True
diaper	4	67%	True
milk	4	67%	True
beer	3	50%	False
cola	2	33%	False
eggs	2	33%	False

```
Global order: bread → diaper → milk
```

Basket	Filtered and ordered
T1	bread → milk
T2	bread → diaper
T3	diaper → milk
T4	bread → diaper → milk
T5	bread → diaper → milk
T6	

2. Second scan: construct the FP-tree

Start at the empty root for **each** basket. Follow an existing child and increment its count, or create a child with count 1. Shared prefixes share nodes; equal item names in different branches remain separate nodes.

A node count measures baskets passing through **that prefix**, not the global frequency of its item. Summing all nodes for an item gives its global count. The root below records inserted basket weight for convenience. It is not an item.

The implementation stores a list of nodes per item (a header index), equivalent in purpose to the node-link chains in textbook diagrams. Blue nodes in each snapshot mark the path just inserted.

In [3]:

```
class Node:
    def __init__(self, item=None, parent=None):
        self.item, self.parent = item, parent
        self.count = 0
        self.children = {}

def insert(root, header, path, weight=1):
    root.count += weight
    node, touched = root, {id(root)}
    for item in path:
        if item not in node.children:
            node.children[item] = Node(item, node)
            header[item].append(node.children[item])
        node = node.children[item]
        node.count += weight
        touched.add(id(node))
    return touched

def draw_tree(root, title, highlight=()):
    # Inline SVG keeps diagrams portable and avoids extra dependencies.
    positions, nodes, edges = {}, [], []
    leaf_index = [0]
    def visit(node, depth):
        nodes.append(node)
        child_x = []
        for child in node.children.values():
            visit(child, depth+1)
            edges.append((node, child))
            child_x.append(positions[id(child)][0])
        if child_x:
            x = sum(child_x)/len(child_x)
        else:
            x = 85 + 155*leaf_index[0]
            leaf_index[0] += 1
        positions[id(node)] = (x, 80+90*depth)
    visit(root,0)
    width = max(360, leaf_index[0]*155+20)
    height = max(y for x,y in positions.values())+50
    svg = [#f'<svg xmlns="http://www.w3.org/2000/svg" width="{width}" height
           f'<svg xmlns="http://www.w3.org/2000/svg" '
           f'viewBox="0 0 {width} {height}" '
           f'preserveAspectRatio="xMidYMid meet" '
           f'style="display:block; width:100%; max-width:{width}px; height:a
           f'role="img">',
           f'<title>{escape(title)}</title><rect width="100%" height="100%"
           f'<text x="15" y="26" font-family="sans-serif" font-size="15">{es
    for a,b in edges:
        x1,y1=positions[id(a)]; x2,y2=positions[id(b)]
        svg.append(f'<line x1="{x1}" y1="{y1}" x2="{x2}" y2="{y2}" stroke="#
    for node in nodes:
```

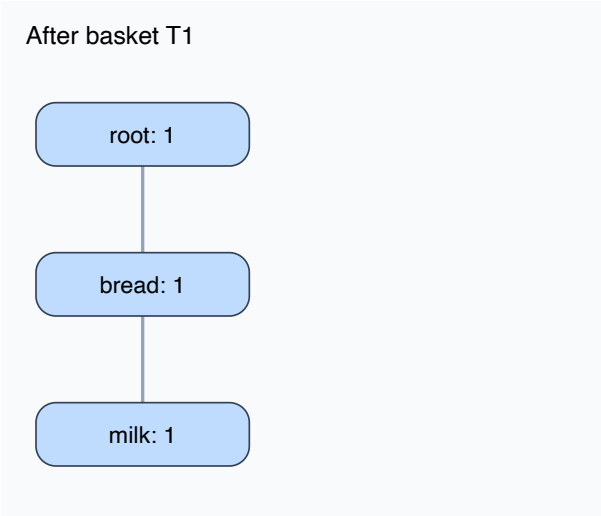
```
        x,y=positions[id(node)]
        color = '#bfdbfe' if id(node) in highlight else '#ffffff'
        text = f'{node.item or "root"}: {node.count}'
        svg.append(f'<rect x="{x-64}" y="{y-19}" width="128" height="38" rx=
        svg.append(f'<text x="{x}" y="{y+5}" text-anchor="middle" font-famil
    #display(SVG(''.join(svg)+'</svg>'))
    #display(SVG(''.join(svg) + '</svg>'))

    # Allow 155 pixels per leaf branch, capped at 800 pixels.
    # A narrow tree stays small; a wide tree can use more space.
    #print(f"Leaf index: {leaf_index[0]}")
    if leaf_index[0] == 1:
        display_width = 300
    else:
        display_width = min(800, leaf_index[0] * 155 + 20)

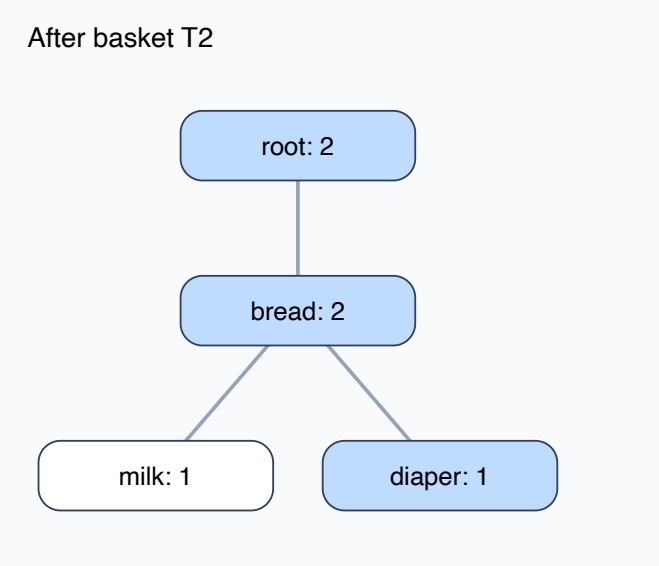
    display(HTML(
        f'<div style="width:{display_width}px; max-width:100%; '
        f'break-inside:avoid;">'
        + ''.join(svg)
        + '</svg></div>'
    ))

root, header = Node(), defaultdict(list)
for i, path in enumerate(ordered_baskets,1):
    touched = insert(root, header, path)
    print(f"Insert T{i}: {' → '.join(path)}")
    draw_tree(root, f"After basket T{i}", touched)
table(["Header item", "Node counts", "Sum = item count"],
      [(x, [node.count for node in header[x]], sum(node.count for node in he
assert all(sum(node.count for node in header[x]) == counts[x] for x in order
```

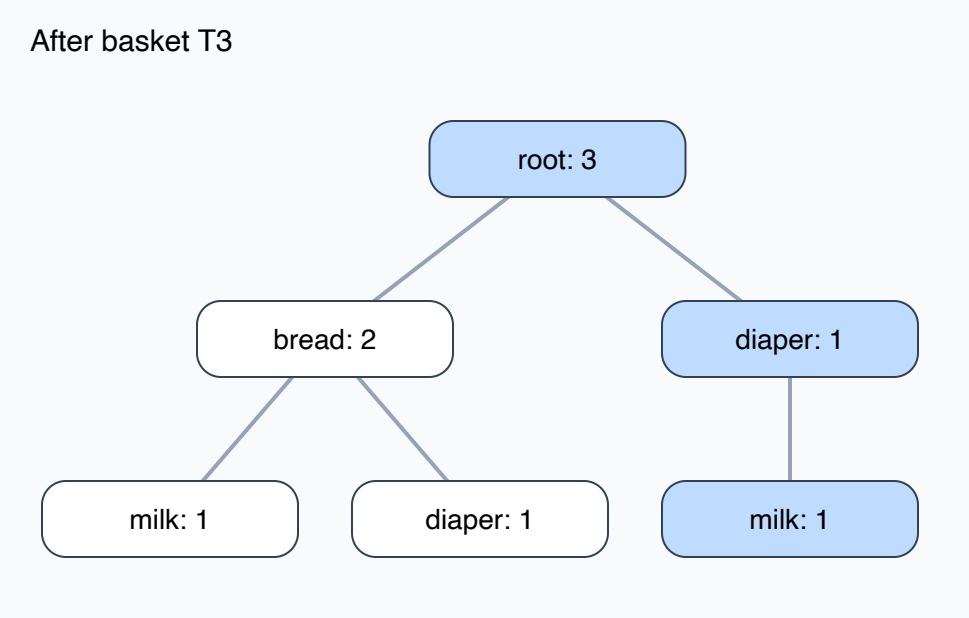
```
Insert T1: bread → milk
```



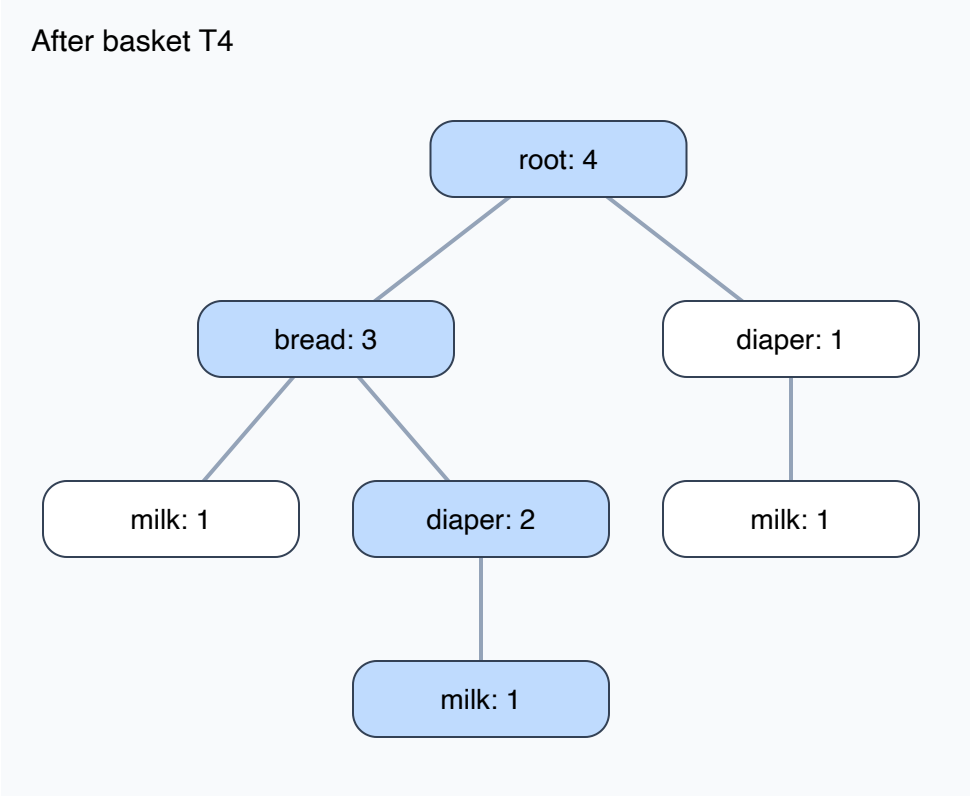

```
Insert T2: bread → diaper
```



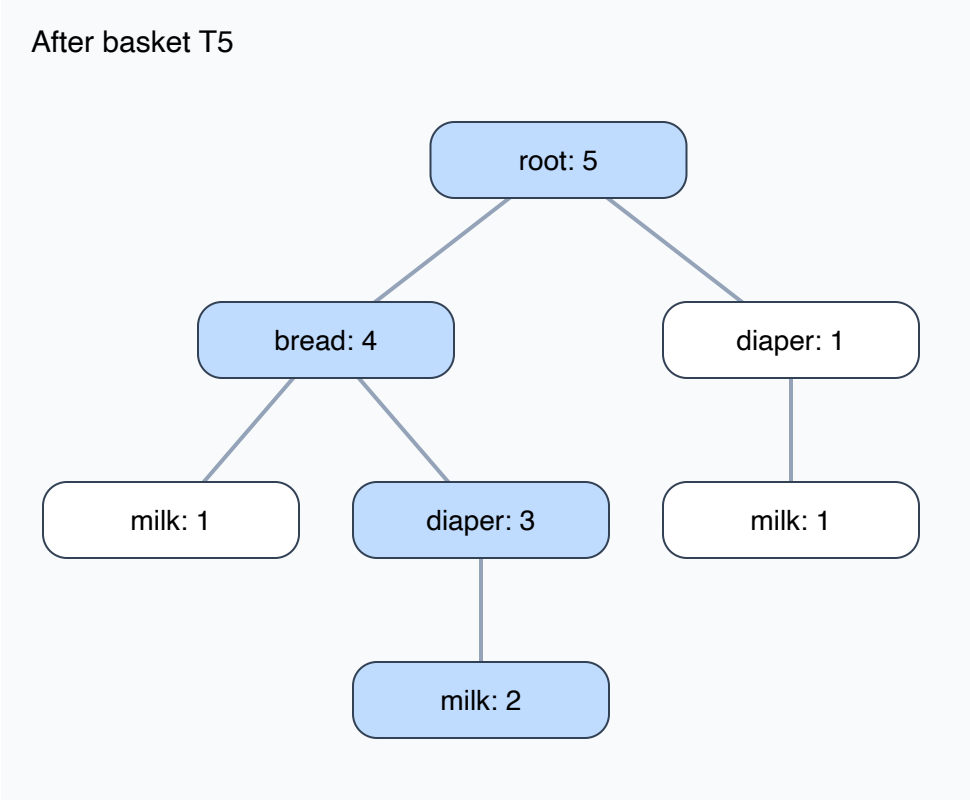
`Insert T3: diaper → milk`



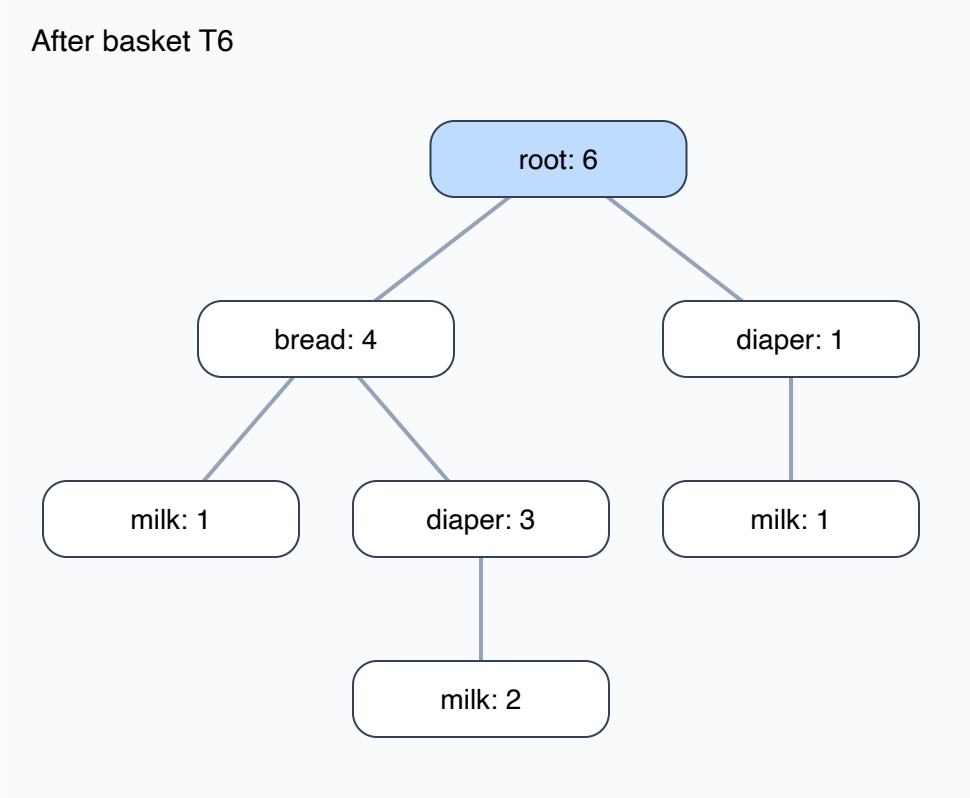
`Insert T4: bread → diaper → milk`




```
Insert T5: bread → diaper → milk
```




```
Insert T6:
```



Header item	Node counts	Sum = item count
bread	[4]	4
diaper	[3, 1]	4
milk	[1, 1, 2]	4

In [4]:
```
display(Markdown(
    "## 3. Conditional pattern bases and conditional trees\n"
    "For a suffix such as **beer**, find every beer node using the header ir
    "\n"
))

if IS_BASELINE:
    display(Markdown(
        "\n"
        "Count items in these weighted paths, keep those meeting the **origi
        "In this dataset beer has count 3, and every beer basket contains di
    ))
```

3. Conditional pattern bases and conditional trees

For a suffix such as **beer**, find every beer node using the header index. Walk upward to obtain its prefix path, excluding both beer and the root. Weight that path by the **beer node's count**, not by an ancestor's count. The collection is beer's conditional pattern base.

In [5]:
```
def conditional_base(nodes):
    base = []
    for node in nodes:
        path, parent = [], node.parent
        while parent is not None and parent.item is not None:
            path.append(parent.item)
            parent = parent.parent
        if path:
            base.append((list(reversed(path)), node.count))
    return base

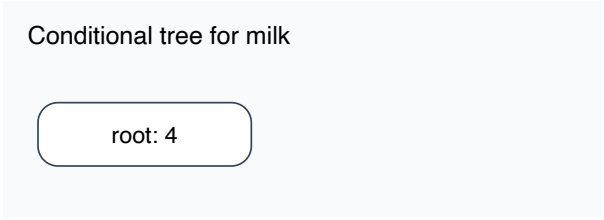
def build_tree(weighted_paths, min_count):
    local_counts = Counter()
    for path, weight in weighted_paths:
        for item in path:
            local_counts[item] += weight
    local_order = sorted((x for x in local_counts if local_counts[x] >= min_
                          key=lambda x: (-local_counts[x], x))
    local_rank = {x:i for i,x in enumerate(local_order)}
    tree, index = Node(), defaultdict(list)
    for path, weight in weighted_paths:
        filtered = sorted((x for x in path if x in local_rank), key=local_ra
        insert(tree, index, filtered, weight)
    return tree, index, local_counts

for suffix in reversed(order):
    base = conditional_base(header[suffix])
    display(Markdown(f"### Suffix: {suffix}"))
    table(["Prefix path (suffix excluded)", "Weight"],
          [(" → ".join(path), w) for path,w in base] or [("No nonempty prefi
    tree, index, local_counts = build_tree(base, MIN_COUNT)
    table(["Conditional item", "Weighted count", "Retained?"],
          [(x, c, c >= MIN_COUNT) for x,c in sorted(local_counts.items())])
    draw_tree(tree, f"Conditional tree for {suffix}")
    if not index:
        print("No frequent extension; this branch stops.")
```

Suffix: milk

Prefix path (suffix excluded)	Weight
bread	1
diaper	1
bread → diaper	2

Conditional item	Weighted count	Retained?
bread	3	False
diaper	3	False




```
No frequent extension; this branch stops.
```

Suffix: diaper

Prefix path (suffix excluded)	Weight
bread	3

Conditional item	Weighted count	Retained?
bread	3	False




```
No frequent extension; this branch stops.
```

Suffix: bread

Prefix path (suffix excluded)	Weight
No nonempty prefix	—

Conditional item	Weighted count	Retained?




```
No frequent extension; this branch stops.
```

Recursively grow patterns

Process suffix items from least to most frequent (with a deterministic tie order). Emit the suffix pattern with its header count, then repeat on its conditional tree. The trace below includes **every recursive branch**, its weighted prefix base, and retained conditional counts. No candidate itemset list is generated. Conditional databases are still traversed: “two scans” describes the original database used to build the initial tree, not the total amount of work.

In [6]:
```
def fp_growth(transactions, min_count, trace=False):
    tree, index, _ = build_tree([(sorted(t), 1) for t in transactions], min_
```

```
    found, steps = {}, []
    def mine(index, suffix):
        frequencies = {x:sum(node.count for node in nodes) for x,nodes in ir
        for item in sorted(index, key=lambda x: (frequencies[x], x)):
            pattern = suffix | frozenset([item])
            found[pattern] = frequencies[item]
            base = conditional_base(index[item])
            conditional_tree, conditional_index, local = build_tree(base, mi
            if trace:
                steps.append((label(pattern), frequencies[item],
                              '; '.join(' → '.join(p)+f' × {w}' for p,w in b
                              ', '.join(f'{x}: {local[x]}' for x in sorted(c
            if conditional_index:
                mine(conditional_index, pattern)
    mine(index, frozenset())
    return found, steps

fp_result, fp_trace = fp_growth(baskets, MIN_COUNT, trace=True)
table(["Pattern emitted", "Count", "Conditional pattern base", "Retained cor
print(f"FP-Growth found {len(fp_result)} frequent itemsets.")
```

Pattern emitted	Count	Conditional pattern base	Retained conditional counts
{bread}	4	∅	∅
{diaper}	4	bread × 3	∅
{milk}	4	bread × 1; diaper × 1; bread → diaper × 2	∅

```
FP-Growth found 3 frequent itemsets.
```

4. Apriori: join, prune, count

Apriori uses downward closure: **every subset of a frequent itemset must also be frequent**. Consequently, a candidate with any infrequent subset can be discarded before scanning baskets.

Start with singleton candidates C_1 and retain frequent singletons L_1. To construct C_k, join pairs of sets in L_{k-1} whose union has size k, remove duplicates, then prune any union missing a frequent $(k-1)$-subset. Scan the baskets to count surviving candidates and retain L_k.

This readable implementation tests all pairs at each join step; optimized implementations can use common-prefix joins. The trace distinguishes **subset pruning before counting** from **support rejection after counting**.

In [7]:
```
def apriori(transactions, min_count, trace=False):
    candidates = {frozenset([x]) for t in transactions for x in t}
    result, steps, k = {}, [], 1
    joined, pruned = set(candidates), set()
    while candidates:
```

```
        candidate_counts = dict.fromkeys(candidates, 0)
        for t in transactions:  # one full counting pass for this nonempty c
            for c in candidates:
                if c <= t:
                    candidate_counts[c] += 1
        frequent = {c:v for c,v in candidate_counts.items() if v >= min_cour
        result.update(frequent)
        if trace:
            steps.append((k, joined, pruned, candidate_counts, frequent))
        k += 1
        joined = {a | b for a,b in combinations(frequent, 2) if len(a | b) =
        candidates = {c for c in joined if all(frozenset(s) in frequent for
        pruned = joined - candidates
        if trace and joined and not candidates:
            steps.append((k, joined, pruned, {}, {}))
    return result, steps

ap_result, ap_trace = apriori(baskets, MIN_COUNT, trace=True)
for k,joined,pruned,candidate_counts,frequent in ap_trace:
    display(Markdown(f"### Level {k}"))
    print("Joined:", ', '.join(sorted(map(label, joined))) or '⌀')
    print("Subset-pruned before counting:", ', '.join(sorted(map(label, prur
    table(["Candidate", "Count", "Support", "Frequent?"],
          [(label(c), v, f"{v/N:.0%}", c in frequent) for c,v in sorted(canc
print("Stop: no candidates remain.")
print("Database counting passes:", sum(bool(step[3]) for step in ap_trace))
```

Level 1

```
Joined: {beer}, {bread}, {cola}, {diaper}, {eggs}, {milk}
Subset-pruned before counting: ⌀
```

Candidate	Count	Support	Frequent?
{beer}	3	50%	False
{bread}	4	67%	True
{cola}	2	33%	False
{diaper}	4	67%	True
{eggs}	2	33%	False
{milk}	4	67%	True

Level 2

```
Joined: {bread, diaper}, {bread, milk}, {diaper, milk}
Subset-pruned before counting: ⌀
```

Candidate	Count	Support	Frequent?
{bread, diaper}	3	50%	False
{bread, milk}	3	50%	False
{diaper, milk}	3	50%	False

```
Stop: no candidates remain.
Database counting passes: 2
```

In [8]:
```
if IS_BASELINE:
    display(Markdown("At level 3, `{beer, bread, diaper}` and `{beer, diaper
```

5. Side-by-side correctness check

Both algorithms should return identical itemsets **and counts**. An independent exhaustive oracle enumerates every nonempty subset of the item universe and counts directly. Exhaustive enumeration is feasible here (only six items), but scales exponentially.

The tests also cover empty input, repeated baskets, a single shared path, disconnected items, thresholds above the dataset size, and reproducible random tiny datasets. These checks support this educational implementation; they are not a proof for every possible input.

In [9]:
```
def brute_force(transactions, min_count):
    universe = sorted(set().union(*transactions))
    found = {}
    for k in range(1,len(universe)+1):
        for subset in combinations(universe,k):
            itemset = frozenset(subset)
            count = sum(itemset <= t for t in transactions)
            if count >= min_count:
                found[itemset] = count
    return found

oracle = brute_force(baskets, MIN_COUNT)
assert ap_result == fp_result == oracle
keys = sorted(oracle, key=lambda s:(len(s), sorted(s)))
table(["Itemset", "Apriori count", "FP-Growth count", "Oracle count", "Suppo
      [(label(s), ap_result[s], fp_result[s], oracle[s], f"{oracle[s]/N:.0%}

# assert len(keys) == 8 # This is only valid with the original exercice. The
#assert oracle[frozenset(["beer","diaper"])] == 3 # This is only valid with
rng = random.Random(42)
cases = [[], [set()], [{"a"}], [{"a","b"}]*4, [{"a"},{"b"}],
         [{"a","b","c"},{"a","b"},{"a"}]]
cases += [[{x for x in 'abcde' if rng.random()<0.5} for _ in range(rng.randi
checks = 0
for transactions in cases:
    for threshold in range(1,len(transactions)+2):
        expected = brute_force(transactions, threshold)
        assert apriori(transactions, threshold)[0] == expected
        assert fp_growth(transactions, threshold)[0] == expected
        checks += 1
print(f"PASS: exact counts agree for the lesson dataset and {checks} additio
```

Itemset	Apriori count	FP-Growth count	Oracle count	Support	Match
{bread}	4	4	4	67%	✓
{diaper}	4	4	4	67%	✓
{milk}	4	4	4	67%	✓

```
PASS: exact counts agree for the lesson dataset and 235 additional dataset/t
hreshold checks.
```

In [10]:

```
r'''
## 6. From itemsets to association rules

Mining itemsets and generating rules are separate stages. For disjoint nonem

- **Support** of $A\to B$: $count(A\cup B)/N$.
- **Confidence**: $count(A\cup B)/count(A)$, the observed fraction of A-bask
- **Lift**: $confidence(A\to B)/support(B)$. A lift of 1 corresponds to inde

Generate all nonempty proper antecedents of each frequent itemset, then filt
'''

display(Markdown(f"""
## *6. From itemsets to association rules*

Mining itemsets and generating rules are separate stages. For disjoint nonem

- **Support** of $A \\to B$: $count(A \\cup B)/N$.
- **Confidence**: $count(A \\cup B)/count(A)$, the observed fraction of A-ba
- **Lift**: $confidence(A \\to B)/support(B)$. A lift of 1 corresponds to ir

Generate all nonempty proper antecedents of each frequent itemset, then filt

Rules are directional; itemsets are not. Associations are not causal, and fi
"""))
```

6. From itemsets to association rules

Mining itemsets and generating rules are separate stages. For disjoint nonempty sets A and B:

- **Support** of $A \to B$: $count(A \cup B)/N$.
- **Confidence**: $count(A \cup B)/count(A)$, the observed fraction of A-baskets also containing B.
- **Lift**: $confidence(A \to B)/support(B)$. A lift of 1 corresponds to independence in the observed frequencies; above 1 indicates positive association.

Generate all nonempty proper antecedents of each frequent itemset, then filter by confidence ≥ 75%. The minimum support was already enforced during itemset mining.

Rules are directional; itemsets are not. Associations are not causal, and five baskets provide very weak statistical evidence.

In [11]:
```
def association_rules(frequent, n, min_confidence):
    rules = []
    for union, union_count in frequent.items():
        for size in range(1,len(union)):
            for antecedent in combinations(sorted(union), size):
                a = frozenset(antecedent)
                b = union - a
                confidence = union_count/frequent[a]
                if confidence >= min_confidence:
                    rules.append((label(a), label(b), union_count/n, confide
                                  confidence/(frequent[b]/n)))
    return sorted(rules)

rules_ap = association_rules(ap_result, N, MIN_CONFIDENCE)
rules_fp = association_rules(fp_result, N, MIN_CONFIDENCE)
assert rules_ap == rules_fp

# assert len(rules_ap) == 8 # This is only valid with the original exercice.
table(["Antecedent →", "Consequent", "Support", "Confidence", "Lift"],
      [(a,b,f"{s:.0%}",f"{c:.0%}",f"{l:.4f}") for a,b,s,c,l in rules_ap])
```

Antecedent →	Consequent	Support	Confidence	Lift

In [12]:
```
# Compare the rules produced by both algorithms.
assert rules_ap == rules_fp, "The algorithms produced different rules!"

# Independently calculate metrics directly from the baskets.
def check_rule(a, b, support, confidence, lift):
    # Count baskets containing every item in both a and b.
    # a | b combines the two sets; <= checks whether all items are in the ba
    union_count = sum((a | b) <= basket for basket in baskets)

    # Count baskets containing every item in the antecedent (a).
    a_count = sum(a <= basket for basket in baskets)
```

```
    # Count baskets containing every item in the consequent (b).
    b_count = sum(b <= basket for basket in baskets)

    expected_support = union_count / N
    expected_confidence = union_count / a_count
    expected_lift = expected_confidence / (b_count / N)

    assert isclose(support, expected_support), "Support mismatch"
    assert isclose(confidence, expected_confidence), "Confidence mismatch"
    assert isclose(lift, expected_lift), "Lift mismatch"

# Map the displayed labels back to their original itemsets.
itemsets_by_label = {label(s): s for s in ap_result}

for a_label, b_label, support, confidence, lift in rules_ap:
    a = itemsets_by_label[a_label]
    b = itemsets_by_label[b_label]

    check_rule(a, b, support, confidence, lift)

    print(
        f"{a_label} → {b_label}:\t"
        f"support = {support:.1%};\t"
        f"confidence = {confidence:.1%};\t"
        f"lift = {lift:.3f}"
    )

print(
    f"PASS: both algorithms produced the same {len(rules_ap)} rules. "
    "All returned rule metrics match direct basket counts."
)

if not rules_ap:
    print("No rules meet the current support and confidence thresholds.")
```

```
PASS: both algorithms produced the same 0 rules. All returned rule metrics m
atch direct basket counts.
No rules meet the current support and confidence thresholds.
```

7. Runtime: measure, then interpret cautiously

Each timed call mines all frequent itemsets from the same in-memory baskets with the same threshold. FP-Growth timings include initial tree construction and recursive mining; Apriori timings include candidate generation and counting. Both exclude imports, visualization, tracing, rule generation, and correctness tests. Both return complete dictionaries.

We warm up both functions and alternate measurement order across nine batches. Each batch averages multiple calls; the median and observed batch range summarize noise, not confidence intervals. A second case repeats the original dataset 200 times,

preserving proportions and patterns while increasing input size. Repetition is **not** a realistic scalability benchmark: it adds no new item variety or basket structure.

In [13]:
```
def benchmark(data, threshold, calls, batches=9):
    functions = {"Apriori": apriori, "FP-Growth": fp_growth}
    samples = {name:[] for name in functions}
    for fn in functions.values():
        fn(data,threshold)
    assert apriori(data,threshold)[0] == fp_growth(data,threshold)[0]
    for batch in range(batches):
        names = list(functions)
        if batch % 2:
            names.reverse()
        for name in names:
            start = perf_counter_ns()
            for _ in range(calls):
                functions[name](data,threshold)
            samples[name].append((perf_counter_ns()-start)/calls/1e6)
    return samples

print("Python:", sys.version.split()[0], "| Platform:", platform.platform())

repeated_baskets = baskets * basket_multiplier
repeated_min_count = ceil(MIN_SUPPORT * len(repeated_baskets))
results = []

# Each tuple contains:
# (dataset label, baskets, minimum count, calls per timing batch)
#
# calls: number of complete algorithm runs within each timing batch.
# The batch's elapsed time is divided by calls to estimate time per run.
# Use more calls for tiny datasets (200) to reduce timing noise,
# and fewer for larger datasets (15) to keep the benchmark manageable.
for name, data, threshold, calls in [
    (f"Original: {len(baskets)} baskets",
     baskets, MIN_COUNT, 200),

    (f"Repeated: {len(repeated_baskets)} baskets",
     repeated_baskets, repeated_min_count, 15),
]:
    samples = benchmark(data,threshold,calls)
    for algorithm, times in samples.items():
        results.append((
            name,
            algorithm,
            median(times),
            min(times),
            max(times),
        ))

table(["Dataset", "Algorithm", "Median ms/call", "Min batch ms/call", "Max b
      [(d,a,f"{m:.4f}",f"{lo:.4f}",f"{hi:.4f}") for d,a,m,lo,hi in results])
# Simple inline bar chart; labels retain exact units.
maximum = max(row[2] for row in results)
chart = '<h4>Median mining time (shorter is faster)</h4>'
```

```
for dataset,algorithm,value,_,_ in results:
    color = '#2563eb' if algorithm == 'Apriori' else '#0d9488'
    chart += f'<div style="margin:10px 0">{escape(dataset)} — {algorithm}: {
display(HTML(chart))
```

```
Python: 3.12.3 | Platform: macOS-27.0.1-arm64-arm-64bit
```

Dataset	Algorithm	Median ms/call	Min batch ms/call	Max batch ms/call
Original: 6 baskets	Apriori	0.0090	0.0089	0.0096
Original: 6 baskets	FP-Growth	0.0230	0.0225	0.0241
Repeated: 1200 baskets	Apriori	1.0673	1.0615	1.0937
Repeated: 1200 baskets	FP-Growth	1.6176	1.5923	2.0197

Median mining time (shorter is faster)

Original: 6 baskets — Apriori: 0.0090 ms

Original: 6 baskets — FP-Growth: 0.0230 ms



Repeated: 1200 baskets — Apriori: 1.0673 ms



Repeated: 1200 baskets — FP-Growth: 1.6176 ms



What the timing can—and cannot—tell you

Tiny inputs can favor Apriori because tree allocation and recursion have overhead. Results depend on the implementation, item universe, transaction density, support threshold, shared prefixes, Python version, and machine load. FP-Growth avoids Apriori's candidate explosion but can still produce exponentially many frequent patterns and can build many conditional trees. Two original-database scans do not mean constant-time mining.

This notebook does not measure memory usage, disk I/O, parallel performance, or optimized library implementations. Do not extrapolate its winner to a real retail workload. For a stronger study, vary basket count, item vocabulary, density, and support independently; use multiple seeded datasets and include peak memory and output size.

8. Try it yourself

1. Change `MIN_SUPPORT` to 0.40 and rerun. More items and triples become frequent. Update or remove the assertions specific to the original 60% example (`MIN_COUNT`

`== 3`, eight itemsets, and eight rules), and treat its narrative examples as fixed reference results.
2. Add a basket containing only eggs. Recompute the denominator and observe how a relative support threshold changes.
3. Raise `MIN_CONFIDENCE` to 1.0. Which rule directions disappear? Update the fixed eight-rule assertion.
4. Explain why a milk node with count 1 can coexist with global milk count 4.

Expected answers for the original example: milk appears under several prefixes, whose counts sum to 4. Beer → diaper has confidence 100%, while diaper → beer has 75%. There are four frequent singletons, four frequent pairs, and no frequent triples.

Question 4 solution: Why can a milk node with count 1 coexist with global milk count 4?

An FP-tree node counts the baskets that share **the particular prefix ending at that node**. It does not represent the item's frequency across the entire dataset.

In the baseline scenario, milk appears at three different nodes:

Path to milk	Node count	Supporting baskets
bread → milk	1	T1
diaper → milk	1	T3
bread → diaper → milk	2	T4, T5

The global milk count is the sum of these node counts:

Global milk count = 1 + 1 + 2 = 4

Therefore, a milk node with count **1** means that one basket follows that particular prefix. Across all prefixes, **four baskets contain milk**, giving milk a support of **4/5 = 80%**.

Sources and implementation notes

The market baskets are embedded illustrative data. Both algorithms here are transparent Python implementations, not wrappers around a mining library. Transactions must be sets of comparable string item names; `min_count` must be a positive integer. FP-Growth uses weighted conditional trees without the optional single-path optimization.

- Agrawal & Srikant (1994), Fast Algorithms for Mining Association Rules: Apriori and the frequent-subset pruning property.
- Han, Pei & Yin (2000), Mining Frequent Patterns without Candidate Generation: FP-trees and recursive conditional-pattern mining.

The diagrams are prefix-count data structures, not supervised decision trees.